

10K TPS Capacity Plan

ต้องใช้ pod และ resource เท่าไรถึงจะรับ **10,000 requests/วินาที** โดยที่ทั้ง API และ database ใช้ทรัพยากร**ไม่เกิน 60 % ของ limit** คำตอบมาจากการวัดจริงแบบขั้นบันไดที่ 1,000 ถึง 4,000 TPS แล้วคำนวณต่อ ไม่ใช่การเดา ทั้งเป้า 10,000 TPS และเพดาน 60 % เป็นค่าตัวอย่างที่ผู้เขียนกำหนดขึ้นเองเพื่อสาธิตวิธีคิด capacity ไม่ได้มาจากโจทย์ วิธีเดียวกันใช้กับเป้าอื่นได้โดยไม่ต้องเปลี่ยนอะไร

วัดจริงถึง	REQUESTS	ล้มเหลว	P95
4,000 TPS	579,999	0	1.12 ms
เพดานที่ตั้ง	หลักฐาน		
60 %	c1-capacity		

01 คำตอบ

API 4 pods, Postgres 1 pod 6 cores / 3Gi

CONFIG ที่แนะนำสำหรับ 10,000 TPS					
ส่วนประกอบ	REPLICAS	CPU LIMIT	MEMORY LIMIT	CPU ที่ใช้จริง	MEMORY ที่ใช้จริง
API (Gin + pgx) scale out ไม่ใช่ scale up	4	500m ต่อ pod รวม 2,000m	128Mi ต่อ pod	1,044m = 52 %	17 MiB = 13 %
Postgres ตัวเดียว ไม่ ต้อง replica	1	6,000m	3Gi	2,187m = 36 %	1,363 MiB = 44 %
ตั้ง <code>requests</code> ไว้ที่ 60 % ของ limit พอดี เพื่อให้เส้นสีเขียวในกราฟทุก ใบคือเส้นงบประมาณ ถ้าเส้นข้อมูลไม่ข้ามเส้นเขียว แปลว่าผ่านเงื่อนไข					

ตัวเลขที่ต้องตั้งใน manifest ทั้งหมด

API Deployment	Postgres StatefulSet
replicas: 4	replicas: 1
limits: cpu 500m mem 128Mi	limits: cpu 6000m mem 3Gi
requests: cpu 300m mem 77Mi	requests: cpu 3600m mem 1843Mi
env: GOMAXPROCS=1 GOMEMLIMIT=100MiB shared_buffers 640MB max_connections 100	

สรุปสั้นที่สุด

ฝั่ง API ให้เพิ่มจำนวน **pod** จาก 1 เป็น 4 โดยไม่ต้องขยาย pod ให้ใหญ่ขึ้น เพราะ Go ถูกตั้ง `GOMAXPROCS=1` อยู่แล้ว การเพิ่ม CPU ให้ pod เดียวจึงไม่ช่วย ส่วนฝั่ง database ให้ขยาย **pod** เดิม จาก 2 เป็น 6 cores และจาก 1Gi เป็น 3Gi ยังคงเป็น Postgres ตัวเดียว ไม่ต้องทำ replication

02 วิเคราะห์คำตอบ

วัดที่อัตราต่ำ แล้วคำนวณขึ้นไป ดีกว่ายิ่ง 10,000 TPS จริง

ถ้ายิ่ง 10,000 TPS จากเครื่องเดียวกับที่รัน cluster อยู่ ตัว k6 เองจะกิน CPU หลาย core ไปแย่งกับ pod ที่เรากำลังวัด ตัวเลขที่ได้จะไม่ใช่ต้นทุนจริงของระบบ แต่เป็นต้นทุนของระบบที่ถูกแย่ง CPU วิธีที่ให้คำตอบแม่นยำกว่าคือ

1. ยิ่งเป็นขั้นบันไดที่อัตราซึ่งเครื่องรับไหวสบายๆ คือ 1,000 → 2,000 → 3,000 → 4,000 TPS ขั้นละ 60 วินาที
2. วัด CPU ที่ใช้จริงในแต่ละขั้น แล้วดูว่าต้นทุนต่อ **1 TPS** คงที่หรือไม่
3. ถ้าคงที่ แปลว่าความสัมพันธ์เป็นเส้นตรง จึงลากเส้นต่อไปที่ 10,000 TPS ได้อย่างมีเหตุผล
4. ตรวจว่าค่าที่คำนวณได้ยังอยู่ได้เพดาน 60 % และตรวจข้อจำกัดอื่นที่ CPU ไม่ครอบคลุมด้วย

จุดสำคัญของการออกแบบ TEST

ใช้ executor แบบ `ramping-arrival-rate` ไม่ใช่การกำหนดจำนวน VU เพราะ **VU ไม่ใช่ TPS** ถ้ากำหนด VU แล้วระบบช้าลง TPS จะลดตามเองโดยที่เราไม่รู้ตัว แต่ arrival rate คือการบังคับอัตราค่าขอต่อวินาทีโดยตรง ถ้าระบบตามไม่ทัน k6 จะรายงาน `dropped_iterations` ให้เห็นชัดเจน ในรอบนี้ยังไป 579,999 ครั้ง จากที่ควรได้ 580,000 คิดเป็น **100.00 %** แปลว่าอัตราที่ตั้งไว้เกิดขึ้นจริงเกือบสมบูรณ์

ส่วนผสมของ traffic ตั้งไว้ที่ 90 % เป็นการค้นหา และ 10 % เป็นการจองแล้วคืน การคืนของทันทีทำให้ของในระบบไม่หมด จึงวัดต้นทุนในสภาวะปกติ ไม่ใช่สภาวะของใกล้หมด

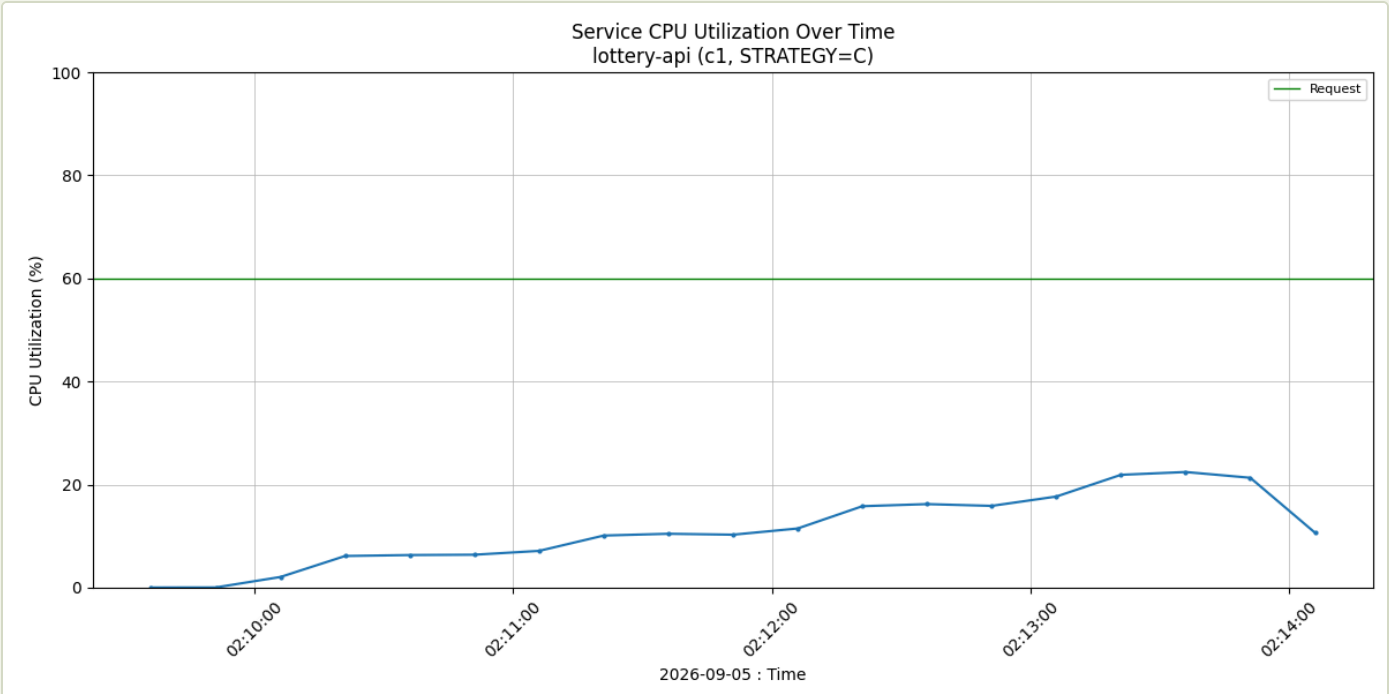
03 ผลวัดขั้นบันได

4 ขั้น บน config ที่เสนอจริง

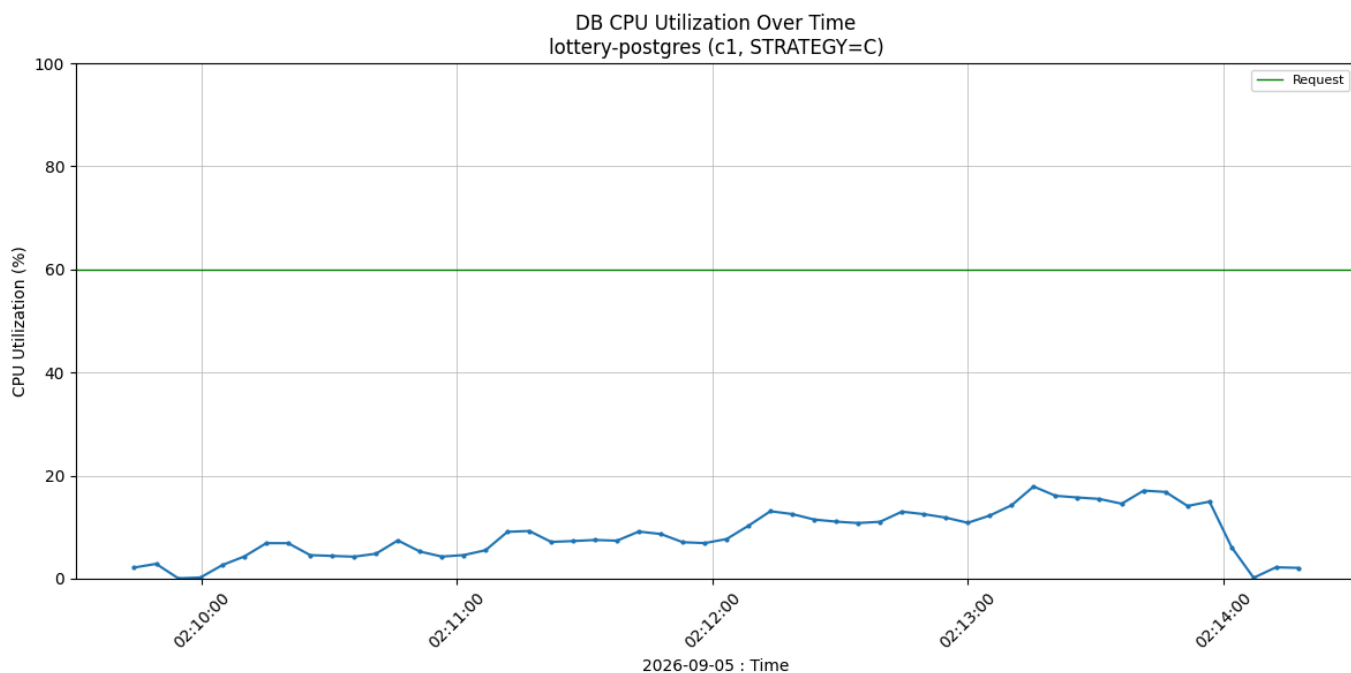
วัดบน API 4 pods และ Postgres 6 cores ตามที่เสนอ ทุกขั้นวัดเฉพาะช่วง 50 วินาทีที่อัตรานิ่งแล้ว ไม่นับ 10 วินาทีแรกที่กำลังไต่ขึ้น

TPS	API ใช้	API % ของ LIMIT	DB ใช้	DB % ของ LIMIT	DB WORKING SET
1,000	100m	5.0 %	320m	5.3 %	876 MiB
2,000	192m	9.6 %	484m	8.1 %	1,109 MiB
3,000	293m	14.6 %	706m	11.8 %	1,283 MiB
4,000	415m	20.7 %	942m	15.7 %	1,323 MiB

latency ตลอดทั้งรอบ p95 = 1.12 ms และไม่มี request ล้มเหลวเลยสักครั้ง



CPU ของ API เป็นขั้นบันได 4 ขั้นชัดเจน แต่ละขั้นคือ 1,000 TPS ที่เพิ่มขึ้น ความสูงของขั้นที่เท่ากันคือหลักฐานตรงๆ ว่าต้นทุนต่อ TPS คงที่



Postgres ก็เป็นชั้นบันไดเหมือนกัน แต่เริ่มจากฐานที่ไม่ใช่ศูนย์ เพราะมีงานพื้นหลังของ database เองอยู่ราว 112m ตลอดเวลา

● DOCS/EVIDENCE/C1-CAPACITY/10-K6.LOG

✓ status 200

```
checks.....: 100.00% ✓ 579999      ✗ 0
data_received.....: 403 MB  1.7 MB/s
data_sent.....: 96 MB   398 kB/s
✓ dropped_iterations.....: 0      0/s
```

running (4m00.0s), 0002/0550 VUs, 550841 complete and 0 interrupted iterations

THRESHOLD

รอบนี้ไม่มี `dropped_iterations` เลย k6 ตามอัตราที่สั่งได้ครบทุกชั้น และฝั่ง server ไม่มี request ล้มเหลวแม้แต่ครั้งเดียว

04 โมเดลและการคำนวณ

แยกงานพื้นหลังออกจากต้นทุนต่อ request

ถ้าเอา CPU ทั้งหมดหารด้วย TPS ตรงๆ จะได้ตัวเลขที่ลดลงเรื่อยๆ ตามโหลดที่เพิ่ม เพราะมีงานพื้นหลังที่คงที่ป้อนอยู่ ต้องแยกสองส่วนนี้ออกจากกันก่อน โดยลากเส้นตรงผ่านจุดที่ต่ำสุดกับสูงสุด

CPU ที่ใช้ = งานพื้นหลัง + ต้นทุนต่อ request x TPS

API = -5m + 0.1049 m/TPS ที่ 10,000 TPS -> **1,044m**

DB = 112m + 0.2075 m/TPS ที่ 10,000 TPS -> **2,187m**

ตรวจเพดาน 60 %

API 1,044m / (4 pods x 500m) = **52.2 %** ผ่าน

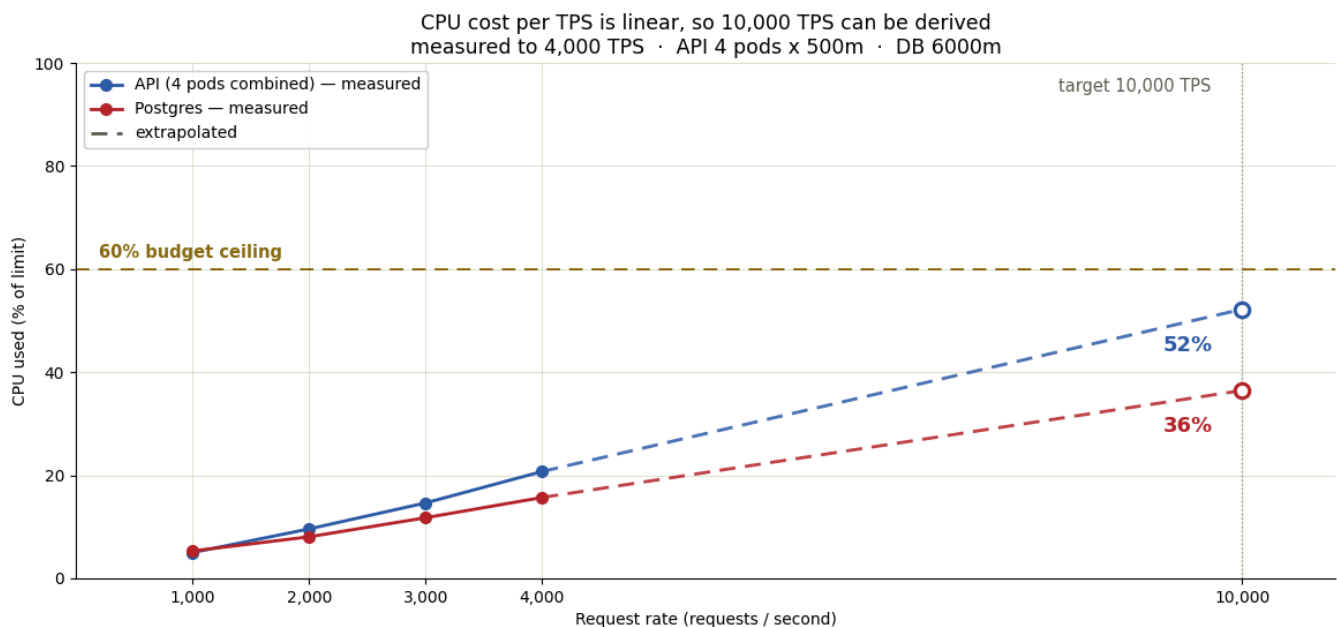
DB 2,187m / 6,000m = **36.5 %** ผ่าน

limit ขั้นต่ำที่ยังผ่านเพดาน

API 1,044m / 0.60 = 1,740m -> ต้องมีอย่างน้อย 4 pods x 500m = 2,000m

DB 2,187m / 0.60 = 3,645m -> ต้องมีอย่างน้อย 4,000m (จะได้ 54.7 % ซึ่งขีดเพดานเกินไป)

ค่าตัดแกนของ API ติดลบเล็กน้อย (-5m) เป็นผลข้างเคียงของการลากเส้นผ่านสองจุด ความหมายจริงคือ API แทบไม่มีงานพื้นหลังเลย ต้นทุนเกือบทั้งหมดแปรตามจำนวน request



เส้นทึบคือของจริง เส้นประคือการคำนวณต่อ เส้นประแนวอนคือเพดาน 60 % ที่ 10,000 TPS ทั้งสองเส้นยังอยู่ใต้เพดาน โดย API อยู่ที่ 52 % และ Postgres อยู่ที่ 36 % แกนตั้งเป็นเปอร์เซ็นต์ของ limit ของแต่ละตัว จึงเทียบกับเส้นงบเดียวกันได้ทั้งคู่

ทำไม API ถึงเป็นตัวที่ขีดเพดานมากกว่า ทั้งที่ database มักเป็นคอขวดในระบบทั่วไป คำตอบอยู่ในรายงานผลทดสอบชุดก่อน คือ Strategy C ย้ายงานหนักออกจาก database ไปแล้ว งานที่เหลือของ

database ต่อหนึ่ง request คือการอ่าน index ไม่กี่ครั้ง ส่วน API ต้องทำงาน HTTP, JSON และ connection pool ซึ่งกลายเป็นส่วนที่แพงกว่า

05 ข้อจำกัดอื่น

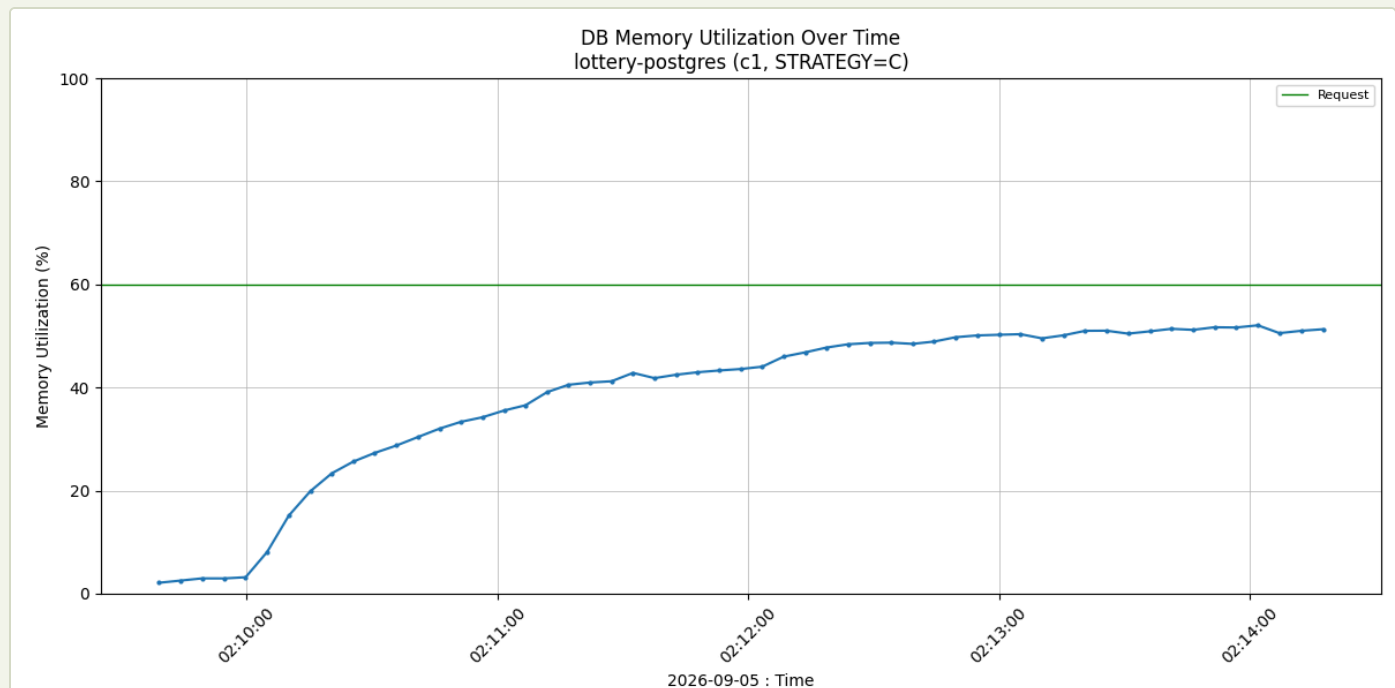
สามอย่างที่โมเดล CPU ไม่ครอบคลุม

1. Memory ของ database ไม่ได้โตเป็นเส้นตรง

ถ้าใช้วิธีลากเส้นตรงกับ memory จะได้คำตอบที่ผิด เพราะ memory ของ Postgres ไม่ได้โตตามจำนวน request แต่โตตามปริมาณข้อมูลที่ถูกดึงเข้า cache ซึ่งจะอึดตัวเมื่อข้อมูลร้อนถูก cache ครบแล้ว ดูจากส่วนต่างของแต่ละชั้นจะเห็นว่ามันลดลงเรื่อยๆ

working set ต่อชั้น	876	->	1,109	->	1,283	->	1,323 MiB
ส่วนต่าง			+232		+174		+41 (ลดลงเรื่อยๆทุกชั้น)
คาดว่าอึดตัวที่			~1,363 MiB		ไม่ใช้การลากเส้นตรงซึ่งจะได้ราว		2,217 MiB

ที่ limit 2,560Mi	->	53.2 %	ผ่านแบบเฉียดฉิว
ที่ limit 3Gi	->	44.4 %	ผ่านสบาย <- เลือกอันนี้



memory ได้ขึ้นแล้วเริ่มแบน รูปทรงนี้คือการเติม cache จนอึด ไม่ใช่ memory leak ถ้าเป็น leak เส้นจะได้ขึ้นเรื่อยๆ ในอัตราคงที่โดยไม่มีที่ที่จะแบน

2. Connection pool

จำนวน connection ที่ต้องใช้พร้อมกันคำนวณจาก *TPS คูณเวลาที่ใช้ต่อ request* ไม่ได้ขึ้นกับจำนวนผู้ใช้

เวลาเฉลี่ยฝั่ง server = 0.69 ms

ที่ 10,000 TPS -> $10,000 \times 0.00069 = 6.9 \text{ connection}$ ที่ต้องใช้พร้อมกัน

ที่มีอยู่ -> 4 pods x pgxpool 10 = 40 connection เหลือเพื่อ

max_connections ของ Postgres ตั้งไว้ 100

3. ขนาดของ node

รวม limit ทั้งหมดของ config ที่เสนอคือ CPU 8,000m และ memory 3,584Mi ในการวัดครั้งนี้ทำบน docker-desktop ที่มี 14 cores แต่มี memory ให้ Kubernetes ใช้ได้เพียง 3.73GiB จึงต้องบีบ database ลงเหลือ 2,560Mi ทอนทดสอบ ซึ่งยังผ่านเพดานที่ 51.7 % บน **production** ควรใช้ 3Gi ตามที่เสนอ เพื่อให้มีระยะห่างจากเพดานมากกว่านี้

06 ถ้าต้องโตกว่านี้

ขยายทางไหนก่อน

เป้าหมาย	API PODS	API ใช้	DB CPU LIMIT	DB ใช้
10,000 TPS ที่เสนอ	4	52 %	6,000m	36 %
15,000 TPS	6	52 %	6,000m	54 %
20,000 TPS	7	60 %	8,000m	53 %
ตัวเลขจากสูตรเดียวกัน ยังไม่ได้วัดจริง ใช้เป็นแนวทางวางแผนเท่านั้น				

ฝั่ง API ขยายง่ายเพราะไม่มี state เพิ่ม pod แล้วได้ throughput เพิ่มตามสัดส่วน ส่วนฝั่ง database จะไปชนเพดานของเครื่องเดียวก่อน จุดนั้นค่อยพิจารณา read replica สำหรับ `/v1/search` ซึ่งเป็น read อย่างเดียว โดยให้ `/v1/allocate` ยังยิงเข้า primary เท่านั้น เพราะ `SKIP LOCKED` ต้องการความถูกต้องระดับ transaction

อย่าเพิ่ง SCALE UP API POD

การเพิ่ม CPU ให้ API pod เดียวจาก 500m เป็น 1000m จะแทบไม่ช่วยอะไร เพราะตั้ง `GOMAXPROCS=1` ไว้ Go จะใช้ได้แค่ thread เดียวอยู่ดี ถ้าจะขยายแนวตั้งต้องเพิ่ม `GOMAXPROCS` ตามด้วย แต่การเพิ่มจำนวน pod ง่ายกว่า และได้ความทนทานเพิ่มจากการมีหลาย instance ด้วย

07 ตอบโจทย์

เชื่อมกลับไปที่คำถามทั้ง 4 ข้อ

คำถามจาก โจทย์	
ใช้ database อะไร	PostgreSQL ตัวเดียว 6 cores / 3Gi รับ 10
algorithm และ index	Strategy C คือค้นเลขที่เข้า pattern จากตาราง 1 ล้านแถวก่อน โดยแปลงหลักหน้าที่ล็อกไว้เป็นช่วงบน pri
กันผลลัพธ์ ซ้ำ	<code>FOR UPDATE SKIP LOCKED</code> ในคำสั่งเดียว พ
รองรับ 10M+ และ โหลดสูง	ต้นทุนการค้นหาไม่ขึ้นกับจำนวนสลาก และต้นทุนต่อ TPS เป็นเส้นตรง จึงวางแผน

08 ข้อควรระวัง

สิ่งที่ตัวเลขชุดนี้ยังไม่ได้พิสูจน์

- **10,000 TPS** มาจากการคำนวณ ไม่ได้จริง ค่าที่วัดสูงสุดคือ 4,000 TPS การลากเส้นต่อไปอีก 2.5 เท่าตั้งอยู่บนสมมติฐานว่ายังเป็นเส้นตรงอยู่ ซึ่งข้อมูล 4 จุดที่มีสนับสนุนสมมติฐานนี้ แต่ไม่ได้พิสูจน์ว่า จะจริงตลอดไป ก่อนขึ้น production ควรยิงจริงจากเครื่องแยกที่ไม่ใช่เครื่องเดียวกับ cluster
- ยังไม่ได้ทดสอบสถานะของ ไกล่หมด รอบนี้คืนของทันทีทุกครั้ง ในของจริงถ้า pattern ยอดนิยมถูกจองไปเกือบหมด ต้นทุนต่อ request จะสูงขึ้นมาก ตามที่เห็นในรายงานชุดก่อนที่ latency ขึ้นจาก 75 ms เป็น 510 ms
- ทุกอย่างรันบน **node** เดียว ทั้ง API, database และตัวยิงโหลดแย่ง CPU ชุดเดียวกัน บน production ที่แยกเครื่องกัน ตัวเลขน่าจะดีกว่านี้

- ไม่ได้ทดสอบตอน pod ล่มหรือ rolling update ซึ่งเป็นสถานะที่ capacity ลดลงชั่วคราว ควรเผื่อ pod เพิ่มอีกอย่างน้อย 1 ตัวถ้าต้องการรองรับการอัปเดตแบบไม่มี downtime

ตัวเลขทั้งหมดมาจาก `docs/evidence/c1-capacity/` ซึ่งมี log ของ k6 แบบเต็ม กราฟทรัพยากร 10 รูป ข้อมูลดิบ CSV และไฟล์โมเดล `50-capacity.json` รันซ้ำได้ด้วย `scripts/scale.sh up` ตามด้วย `make capture-c1` แล้วคำนวณใหม่ด้วย `python3 scripts/capacity.py docs/evidence/c1-capacity 10000 0.60` กลับไปใช้ config เดิมของ PoC ได้ด้วย `scripts/scale.sh down`